Integration guide - Llama Desktop Launcher

Version 0.7.0 · English

This guide explains how to connect clients and applications to the server started by the launcher: endpoint, model ID, keys, ready-made setups for OpenCode and other clients, code examples, network and firewall. To install and use the app, see the **User manual** (MANUAL-en). To solve problems, see the **Diagnostics guide** (DIAGNOSTICS-en).

The app interface is in Portuguese; button names are given in Portuguese with a translation where helpful.

1. Overview

All three engines (llama.cpp, vLLM and Strata/PolyStrata) expose an **OpenAI Chat Completions**-compatible API under /v1. The launcher does not create a public internet API: the server only exists while the launcher is open with the server started, and it only listens on the chosen address.

There are three ways to connect:

Mode	Address	Credential	Lifetime
Direct, local	http://127.0.0.1:PORT/v1	None (use local in clients that require the field)	While the server runs
Direct, network	http://LAN-OR-TAILSCALE-IP:PORT/v1	Permanent API key	The key survives restarts until you create a new one
Shared	http://LAN-OR-TAILSCALE-IP:SHARED-PORT/v1	Temporary sharing credential	Until revoked, the server stops or the launcher closes

Do not mix credentials: the permanent key only works on the direct port, and the temporary credential only works on the shared port.

2. Endpoint and ports

Choose the **Endereço de acesso** (access address) before clicking **Iniciar servidor** (Start server). Once the model loads, the **Conectar aplicativo** (Connect application) panel shows a line like this:

API: http://192.168.1.5:8080/v1 · Modelo API: modelo-local · Interface: http://192.168.1.5:8080/

- **Direct API:** the first free port between **8080 and 8180** on the chosen address. **Copiar link da API** copies the address ending in /v1.
- **Shared API:** with LAN or Tailscale, a second port between **8181 and 8280**, shown as **API compartilhada**.
- **Local bridge:** when you click **Abrir no navegador** (Open in browser) with network access, the launcher opens the interface through a 127.0.0.1 port that is already authenticated. It only serves this computer.

Ports are examples: they change if another one is busy. After restarting the server or the computer, check the IP and port and update your clients. On another computer, 127.0.0.1 points to the client itself, not to the server.

The launcher detects 10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16 as **Rede interna** (local network), and 100.64.0.0/10 as **Tailscale**. Public addresses and IPv6 are not listed.

3. Model ID per engine

The client must send the model field. Use the **Modelo API** shown in the panel:

Engine	Model ID	Note
llama.cpp	modelo-local	Fixed, so clients don't change when you switch GGUF files.
vLLM	modelo-local	The launcher passes --served-model-name modelo-local.
Strata / PolyStrata	model_name from the strata-*.json config	Without model_name, it is the file name without strata- and .json.

The setups copied by the launcher already contain the right ID. To switch models, stop the server, select another one and start again. Open clients do not restart their conversation by themselves, and the context limit may have changed.

Check the ID with GET /v1/models (examples in section 7).

4. Keys and credentials

Permanent API key

- It is created on the first network start and stored in server-api-key.txt, in the app data folder.
- **Mostrar chave** shows the key, and **Copiar chave** copies it to the clipboard.
- **Nova chave** creates a new key and invalidates the old one. Stop the server first and update your direct-access clients.
- Clients send Authorization: Bearer YOUR_KEY. The link alone grants no access.
- Store the key in the client's secret store or an environment variable, outside code and repositories. Do not put the key in a URL or in public messages.

Temporary sharing credential

- It is created each time the server starts on LAN or Tailscale and is embedded in **Copiar link com acesso**, **Copiar Bash/PowerShell com acesso** and **Copiar configuração com acesso**.
- **Revogar acesso compartilhado** replaces the credential and drops shared connections. Copy again to grant access again.
- Stopping the server or closing the launcher ends sharing; after a restart the credential is different.
- In the browser, the token leaves the address bar after authentication and is kept in an HttpOnly cookie.
- Revoking sharing does **not** revoke the permanent key or clients that use it.

5. Ready-made client setups

With the server running, choose the client under **Conectar aplicativo**. The launcher does not install clients or overwrite configurations: it copies a command or a configuration snippet for you to paste.

- **Commands** (OpenCode and Aider): **Copiar comando Bash** (Linux/macOS) or **Copiar comando PowerShell** (Windows). Paste into a terminal inside the project folder. On the network, the command asks for the key with hidden input; the key is not in the copied text.

- **Configuration** (all clients): **Copiar configuração**. On the network, replace SUBSTITUA_PELA_CHAVE_DA_API with the key from **Copiar chave**. The **com acesso** (with access) version already carries the temporary credential and the shared endpoint.

The setups use the server context divided by the simultaneous sessions as the per-session context, and the smaller of 1024 and half of that context as the output limit. Copy again if you change the model, address, context or sessions.

Client	How to use the copied setup
OpenCode	The command defines the llama-local provider for that run only through OPENCODE_CONFIG_CONTENT and runs opencode --model llama-local/ID. To make it permanent, merge the copied configuration into the project's opencode.json or ~/.config/opencode/opencode.json.
Aider	The command sets OPENAI_API_BASE and OPENAI_API_KEY for that run only and runs aider --model openai/ID.
Pi	Merge into ~/.pi/agent/models.json, keeping existing providers, and select the llama-local provider's model in /model.
Hermes	Merge the model block into config.yaml and run hermes chat. Current versions need at least 64000 tokens per session for tools; the setup exports the real context without inflating it.
Continue	Merge the models block into config.yaml, keeping existing models.
Cline	In settings, choose **OpenAI Compatible** and fill in Base URL, API Key and Model ID.
Codex	Experimental: requires the Responses API, which has not been validated with these engines. Merge into ~/.codex/config.toml and set LLAMA_API_KEY.
Generic	Base URL, API Key, Model ID, context window and output limit fields for any OpenAI-compatible client.

The client edits files on the machine where it runs; inference happens on the server. Tools (tool calling), vision and quality depend on the model and its template. Don't enable features just because the endpoint says it is OpenAI-compatible.

6. Integrate your own application

Use **Genérico - OpenAI Compatible** for your own app, service, automation or any client that accepts Chat Completions.

1. Choose the address before starting: **Somente este computador** for a client on the same PC, **Rede interna** for devices on the same network, **Tailscale** for devices on your tailnet.
2. Click **Iniciar servidor** and wait for the endpoint. On LAN/Tailscale, open the firewall (section 9).
3. Pick the mode (section 1) and copy the matching setup.
4. Set **Base URL** to the address ending in /v1, **API Key** to the credential and **Model ID** to the ID in the panel. If the client already appends /chat/completions, don't repeat that path.
5. Test GET /v1/models first, then POST /v1/chat/completions with JSON and the Authorization: Bearer KEY header.

Each request's context window is the server context divided by the simultaneous sessions. Prompt, history, tools and answer must fit in it. The API only returns text: running code or editing files is your application's job.

7. Examples

In the examples, the Base URL already ends in /v1, so we only append /models or /chat/completions. Replace modelo-local with the Strata ID when needed. For direct local access, enter the key local.

Linux/macOS with Bash and curl

The key is read without being shown.

```
(
  read -rp 'Base URL (ending in /v1): ' LLAMA_BASE_URL
  read -rsp 'API Key: ' LLAMA_API_KEY
  printf '\n'
  curl --fail-with-body -sS \
    -H "Authorization: Bearer $LLAMA_API_KEY" \
    "${LLAMA_BASE_URL%/}/models"
  printf '\n'
  curl --fail-with-body -sS \
    -H "Authorization: Bearer $LLAMA_API_KEY" \
    -H 'Content-Type: application/json' \
    "${LLAMA_BASE_URL%/}/chat/completions" \
    --data '{"model":"modelo-local","messages":[{"role":"user",
    "content":"Reply only: connected"}],"max_tokens":64,
    "stream":false}'
)
```

The first result lists the model ID. In the second, the answer is in choices[0].message.content.

Windows with PowerShell

```
& {
  $baseUrl = (Read-Host 'Base URL (ending in /v1)').TrimEnd('/')
  $secureKey = Read-Host 'API Key' -AsSecureString
  $apiKey = [System.Net.NetworkCredential]::new('', $secureKey).Password
  $headers = @{ Authorization = "Bearer $apiKey" }
  Invoke-RestMethod -Uri "$baseUrl/models" -Headers $headers
  $body = @{
    model = 'modelo-local'
    messages = @(@{ role = 'user'; content = 'Reply: connected' })
    max_tokens = 64
    stream = $false
  } | ConvertTo-Json -Depth 5
  $response = Invoke-RestMethod -Method Post `
    -Uri "$baseUrl/chat/completions" -Headers $headers `
    -ContentType 'application/json; charset=utf-8' `
    -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
  $response.choices[0].message.content
}
```

Python (standard library)

Set LLAMA_BASE_URL, LLAMA_API_KEY and, if needed, LLAMA_MODEL in the process environment.

```
import json
import os
from urllib.request import Request, urlopen

base_url = os.environ['LLAMA_BASE_URL'].rstrip('/')
api_key = os.environ['LLAMA_API_KEY']
payload = {
    'model': os.environ.get('LLAMA_MODEL', 'modelo-local'),
    'messages': [{'role': 'user', 'content': 'Reply: connected'}],
    'max_tokens': 64,
    'stream': False,
}
request = Request(
```

```
    base_url + '/chat/completions',
    data=json.dumps(payload).encode('utf-8'),
    headers={
        'Authorization': 'Bearer ' + api_key,
        'Content-Type': 'application/json',
    },
    method='POST',
)
with urlopen(request, timeout=120) as response:
    result = json.load(response)
print(result['choices'][0]['message']['content'])
```

The 120-second timeout is an example; large models may need more.

Streaming

With "stream": true, the answer arrives as Server-Sent Events (SSE): data: {...} lines with chunks in choices[0].delta.content, ending with data: [DONE]. Use a library that handles SSE or read line by line. The examples above use complete responses to stay simple.

8. Network: LAN or Tailscale

- **Tailscale** is preferred: traffic is encrypted and only authorized devices on your tailnet reach the server. The client must be on the same tailnet.
- **Local network (LAN)** uses unencrypted HTTP. Use it only on trusted networks; anyone on the network can see the traffic, although they still need the key to use the model.
- The LAN address may change when the router renews DHCP. If you use it often, reserve the IP in the router.

9. Firewall

Windows

The launcher does not touch the firewall when it starts the server. With the server running on LAN or Tailscale, the **Firewall do Windows** section on the **Llama** screen creates the rules:

1. Check the suggested **client range** (for example 192.168.1.0/24 or 100.64.0.0/10). It must contain the chosen address and stay inside a private or Tailscale network.
2. Click **Criar regra para a porta ativa** (Create rule for the active port) and accept the Windows administrator prompt.
3. The launcher creates one inbound TCP rule for the API port and another for the shared port, each limited to the executable, the port and the range.

Engine	Executable allowed on the API port
llama.cpp	llama-server.exe from the engine folder
Strata / PolyStrata	The base python.exe of the .venv environment (read from pyvenv.cfg)
vLLM (Docker)	None: Docker Desktop publishes the port, and the rule must be created manually

The shared port is allowed for the launcher executable. If the port changes, create another rule. Rules are named LlamaDesktopLauncher-… and appear in **Windows Defender Firewall with Advanced Security**, where you can delete them.

For vLLM in Docker, Docker Desktop usually creates its own inbound rule during installation. If clients cannot connect, create a rule for the displayed port, limited to the range, in an administrator PowerShell:

```
New-NetFirewallRule -DisplayName 'vLLM Docker 8080' -Direction Inbound `
  -Action Allow -Protocol TCP -LocalPort 8080 -RemoteAddress 192.168.1.0/24
```

Linux

Open the API port and the shared port only for the client range. Examples for port 8080:

```
sudo ufw allow from 192.168.1.0/24 to any port 8080 proto tcp
sudo firewall-cmd --add-rich-rule='rule family="ipv4" source address="192.168.1.0/24" port port="8080" protocol="tcp" acce
```

Use your distribution's tool (ufw on Ubuntu and derivatives, firewalld on Fedora and RHEL). The firewall-cmd rule above lasts until reboot; add --permanent and run sudo firewall-cmd --reload to keep it.

macOS

If the macOS firewall is on, the system asks whether llama-server may accept connections the first time it listens on the network. Allow it only if you will use network access.

10. Quick check

Result	What to check
Connection refused or timeout	Server started and model loaded, current IP and port, same network or Tailscale up, firewall.
401 or 403	Bearer header present, the right credential for the port (permanent on direct, temporary on shared), sharing not revoked.
404	Base URL ends in /v1, no /v1/v1; the client uses Chat Completions.
400 or context error	Valid JSON, correct model ID, request within the per-session context.
Client doesn't recognize the new model	Check /v1/models, restart the conversation and update the ID, limit and credential.

The **Diagnostics guide** covers each case and the app's messages in detail.